

Comparing FBTSeg and NNTree Implementations with Established Ensemble Methods

Roberto Angelo Fernandes Santos

Departamento de Banco de Dados
Faculdade Impacta de Tecnologia
São Paulo-SP, Brazil.
rafss@hotmail.com

Roberto Souto Maior de Barros

Centro de Informática
Universidade Federal de Pernambuco
Recife-PE, Brazil.
roberto@cin.ufpe.br

Abstract — This paper compares implementations of *FBTSeg*, a recent experimental segmentation method, and of *NNTree*, originally a neural network tree based segmentation method, with traditional methods for the combination of classifiers, namely bagging, boosting, and the traditional segmentation using information gain to split. The tests were carried out using three data mining techniques with distinct characteristics, specifically, linear regression, logistic regression, and multilayer perceptron neural networks, in four artificially built datasets. The datasets design was aimed at understanding the specific circumstances where each method, simple or combined, would present better performance. Results from this experiment suggest that blending classifiers using segmentation is a viable solution to improve the performance of both statistic regressions, that both *FBTSeg* and *NNTree* are in general more predictive than the traditional segmentation, while bagging and boosting are more effective alternatives for improving neural networks models.

Keywords—classification methods; segmentation; data splitting; machine learning.

I. INTRODUCTION

The results of decision models based on data depend on both the quantity and the quality of the information available for modeling. So, adequately predicting future events is a key element in the decision support systems used by modern companies. Choosing the predictive technique (for example, a statistic regression or a neural network) that is more adequate for some specific problem in hand may be an additional factor for improving accuracy [1] [2] [3] and this search goes beyond a single classifier.

In fact, for a while, specialists and researchers have been investigating different combinations of multiple classifiers, with different parameters and sampling methods, looking for the best configuration of ensembles to maximize the performance of the final solutions [4] [5] [6] [7].

More specifically, to combine classifiers means using several of them to create an integrated ensemble learner with better performance than all the individual predictors used to create it.

Ensembles usually improve results when there is good diversity among these learners [8]. Diversity exists when different classifiers present different variances and biases in the training. The combination of predictors is aimed at maximizing

the prediction power or some other performance measure [5]. Diversity can also be created using the same classifier several times, changing, in some way, its parameters or the training set.

Classic methods for combining multiple classifiers include bagging [5] [9], boosting [5] [9], stacking [4] [5], and segmentation (decomposition) [10].

Segmentation is the division of the available dataset in a number of sub-populations and the development of a prediction model for each segment; then, each example is predicted according to the specific model that it belongs [10].

This paper presents the results of experiments comparing implementations of two newer segmentation methods, *FBTSeg* (Find Best Tree Segmentation) [11] – formerly *RISKSEG* [12] – and *NNTree* (Neural Network Tree) [13], with the traditional segmentation using information gain as the metric to choose the best split [14] (here we named it *TradSeg*), as well as bagging and boosting.

These five methods were combined with three base learners and tested using four artificially built datasets, with three sample sizes each.

The three base learners chosen are linear regression (Linear Discriminant Analysis) [15], used to estimate the value of a dichotomic variable (0 or 1); (binary) logistic regression [15], a traditional technique of statistic inference; and multilayer perceptron neural network (MLP), trained with the backpropagation or Levenberg-Marquardt algorithms [16].

The rest of this paper is organized as follows. Our implementations of the three segmentation methods are discussed in Section II. Section III introduces the used datasets. The experiments design and results are described in Section IV. Finally, Section V presents our conclusions and proposes future work.

II. SEGMENTATION METHODS IMPLEMENTATION

The implementation of segmentation methods can be done using different approaches. Many of them rely on subjective strategies, based on previous knowledge of the problem and/or the data by an expert. Usually, they are based on (a) the existence (or absence) of filled information in a number of selected attributes as a criterion for the split and/or (b) the feeling of the expert based on his/her previous experience [14].

In the past, it has been suggested that it might not be possible to create good segmentation structures without the subjective knowledge of an expert [10]. However, methods and algorithms to find adequate segmentation structures have already been proposed. These methods usually represent the segmentation structure as a tree and use exhaustive or semi-exhaustive search algorithms to choose the best segmentation strategy [10].

This work concentrates on objective and systematic approaches. As already mentioned, the segmentation methods chosen for our experiments are: *TradSeg* (the segmentation using information gain), *NNTree*, and *FBTSeg*. In all of them, the result is a tree of models (learners), with disjoint partitions of the training sample.

The implementations of all combination methods have been developed using the language of the SAS Enterprise Miner software version 4.0 [17]. All three base classifiers (MLP, linear regression and logistic regression) algorithms used in the tests were already implemented in the software.

The selection of the base classifier to be used in each run was based on a parameter (*Inductors*) of the aforementioned software. It is important to say the *FBTSeg* implementation supports the use of different base classifiers in different nodes of the same model tree but this possibility was not exploited in the experiments reported here.

A. *NNTree* – Neural Network Tree

An *NNTree* [13] is a decision tree where each non-terminal node has a neural network. Its original implementation uses an MLP to design the classification tree. The method recursively divides the training set trying to create leaves with elements from a single class – presumably, each non-terminal node has elements from more than one class and uses a neural network to separate them.

The process has three main steps: splitting nodes, checking whether the new nodes should also be split, and labeling terminal nodes.

In other words, *NNTrees* are recursive classifiers that partition the training set aiming to obtain nodes with elements of a single class. An example of the classification structure of *NNTree*, adapted from [13], is presented in Figure 1.

Observe in the example that the MLP_1 node generated the terminal node A_{12} , with instances of class II, as well as nodes A_{11} and A_{13} with new neural networks MLP_2 and MLP_3 , respectively. These last two MLPs then generated other terminal nodes (A_{21} , ..., A_{2m} , etc.) as well as other nodes containing MLPs (MLP_4 and MLP_5).

Even though the original implementation only uses neural networks, the version implemented for the experiments reported here is an adapted algorithm to also support both linear and logistic regressions in the non-terminal nodes.

B. *FBTSeg* – Find Best Tree Segmentation

The *FBTSeg* method [11] aims to find a set of segmented models that improve the performance based on a tree built from divisions using attribute classes. It implements an algorithm that selects the best segmentation through combinations of

candidate nodes from a set of preselected attributes (P), which is a parameter. In other words, after the attribute selection, each attribute class is used to split and the best segmentation is chosen to improve the accuracy of the model for this node.

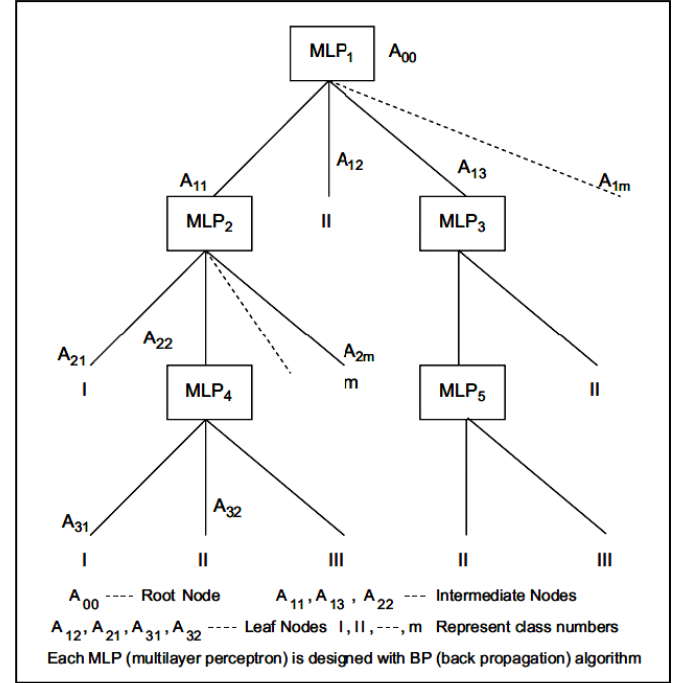


Figure 1. Example of *NNTree*

The current implementation of *FBTSeg* generates, for every node, *two* segments with their specific models which, when combined, might generate better results than the upper node model, originating a binary tree. Thus, the method splits each relevant population in two segments, depending on whether $X_0=0$ or $X_0=1$, where X_0 is a predictive variable. This is equivalent to building a model so that every other variable Y has two variables $Y_0 = (Y | X_0=0)$ and $Y_1 = (Y | X_0=1)$ [14].

So, in all nodes that are candidates for segmentation, the interactions of each (dichotomic) predictive variable of the problem are tested with the other predictive variables (using a statistic regression), building models corresponding to each of them. Therefore, the number of models is the number of variables available for the split. Then, the chosen split is the one corresponding to the model with the best performance, provided this combined segmented model improves the results of the model of the original node.

It is important to point out that the best option to split each node should be chosen based on some specific metric for optimizing the results of the problem, e.g. classification error, mean squared error, KS2 [18], ROC [19], and others.

Additionally, it is worth mentioning that *FBTSeg* can also use categorical variables with more than two classes to find the best segmentation.

Similar to other supervised learning algorithms, *FBTSeg* is prone to *overfitting* and, therefore, it is advisable to create a validation dataset and to apply a cross-validation between the training and validation datasets. In this scenario, the training

dataset is used to train the predictors, while the validation dataset is used to compare the scores among the segments and for choosing the best segmentation.

Algorithm 1, adapted from [12], presents pseudo-code for a possible implementation of *FBTSeg* showing the search for candidate segmentations. The first loop selects the best variable and the second chooses the category of this variable that may be used to split the data. So, after selecting the variable (*Var*) with the best regression results in the interaction with the other variables (factorial), all the possible binary combinations of this variable categories are tested to find the best category (*Cat*) and generate the best pair of segments, one for "*Var = Cat*" and another for "*Var $\neq$ Cat*". After the segmentation, the predictive performance of the suggested pair of segments is compared to that of the upper node. The segmentation is confirmed if it improves the results of the upper node. As in decision trees, the segmentation continues until some stop criterion is reached. The final result is a set of models generated from disjoint subsets of the training data.

The main parameters of *FBTSeg* are:

P: Selected candidate variables. The *FBTSEG* method assumes there is a set *P* of attribute variables available for the segmentations. In other words, the method permits choosing a priori the variables eligible to be used in the segmentations. The base algorithm works with categorical variables.

D: Performance metric to be calculated and evaluated for choosing the best segmentation. Metrics available in the current implementation of *FBTSeg* are: classification error, KS2 [18], ROC [19], and Odds Ratio [14].

f: Combination function or meta-learner. The result of each segmentation possibility needs to be joined, such that *D* can be calculated and its result can be compared to those of the other segmentation possibilities, including the model of the upper node (without segmentation). The *FBTSeg* implementation can use *Stacking* [4] or *Marginal Odds* [14] as the meta-learner.

B: Stop criteria. The available options are minimum number of examples in a segment; minimum gain and maximum loss of performance acceptable in segmentations (percentages); and maximum depth of the segmentation tree. The minimum number of examples in a segment prevents non-representative models from being trained as well as categories without a relevant number of examples from being tested. The maximum depth of the segmentation tree helps limiting the execution time.

FactorialTechnique: Prediction technique used in the selection of candidate variables for the segmentation. We used logistic regression but RNA and SVM [5] could also be used.

NVarTested: Number of variables that will be selected by the prediction technique (*FactorialTechnique*). If the chosen value is 1 (one), only the best variable will be selected, as illustrated in Algorithm 1.

ValidationSize: Size of the validation dataset, defined as a percentage of the training dataset size. A validation dataset is used for the calculation of the performance metric (*D*) as well as in techniques that require it in the training process, such as neural networks, which use it to tackle overfitting.

The *FBTSeg* method has the following steps:

Step 1 (Generate Subsets): the method builds a subset of *P* with *k* categorical variables, using *Factorial Models* to select the best splits according to metric *D*. In other words, the regressions with the best performance indicate the variables that should be chosen for splitting. In theory, the best performance regression indicates *the best variable* for the segmentation. However, *D* might present relevant variances and the results of several regressions might be statistically equivalent.

After the *k* variables are selected, a set $V=\{v_1, v_2, \dots, v_k\}$ is built. Then, for each candidate variable v_i , the split process is run, i.e., the set $C_i=\{c_{i1}, c_{i2}, \dots, c_{in}\}$ of categories of v_i is built and the records of the relevant node are separated. Each split generates two segments, with those records satisfying $v_i = c_{ij}$ and $v_i \neq c_{ij}$, respectively, for all elements of C_i . However, it is important to notice that the generated segments are subject to the stop criterion *minimum number of examples in a segment*.

Step 2 (Generate Classifiers): for each of the segments generated in step 1, a new classifier is built. Although the theory behind the method considers that interactions among the variables are based on statistic regressions, classifiers of the segments can also be built using other regression techniques such as neural networks-

Step 3 (Select Best Segmentation): based on the list of results of the metric *D* for each pair of segments generated, the best result found indicates the variable and category that should be used in the possible segmentation of the node. Note that not all segmentation multi-classifiers will improve the performance of the corresponding upper node original classifier [10].

Step 4 (Recursive Search): after all the terminal nodes in the current level of the tree have passed the split process, and provided no stop criterion has been reached, the segmented nodes created in this level are submitted to the same process, possibly originating a new level in the tree.

Step 5 (Categorization of Numeric Variables): as in decision trees, segmentations can also use numeric variables to create split rules. In decision trees, the algorithm finds the point of a numeric interval that best splits the data based on a metric to separate the values of the target variable, for example, entropy, information gain, chi-squared, etc. [10].

The *FBTSeg* approach is similar, but it searches the point where the split leads to the most distinct data subgroups regarding the target and predictive variables, i.e., the point where the interaction among these variables is greater.

Specifically, the original numeric variable is categorized in *n* parts, generating an equivalent categorical variable that represents it. The best value for *n* can either be defined experimentally or be chosen by an expert with previous experience on the variable distribution.

C. TradSeg – Segmentation by Information Gain

A fairly common strategy to segment is to use the same criteria used in the creation of the nodes of a decision tree, i.e. metrics such as *Information Gain*, *Entropy*, *Kolmogorov-Smirnov (KS)* and others [20].

N – Training set of a given terminal node
 S_1, S_2 – Subsets of N generated by the segmentation
 p – Number of input variables of N
 $X_1, X_2, \dots, X_p$ – Discrete input variables of N
 w – Local variable that stores the number of categories of each discrete variable
 $C[w]$ – Local array that stores the values of each discrete variable
 y – Target variable (dependent)
 $fMet()$ – Performance Function (larger values are considered to be better in this example)
 $fMerge()$ – Merge function to join the results of the models
 Var – Variable chosen to test its categories
 Cat – Category of Var
 $rMod$ – Answers of the factorial models
 $rMod_0$ – Best answers of the models
 $rMod_1, rMod_2$ – Answers of the models of the candidate segments
 $rMod_{12}$ – Joint answer of the models of the candidate segments

Input: N ;

$rMod_0 \leftarrow \emptyset$;

(Best answers initialization)

For $i = 1$ to p do {

(Loop over all the input variables)

$rMod \leftarrow$ Regression answers $y \sim \sum_{j=1}^P X_j + \sum_{j=1, j \neq i}^P X_j X_i$, with data from N ;

(Trains model with interactions)

If $fMet(rMod) > fMet(rMod_0)$ {

(If the model improves the result)

$rMod_0 \leftarrow rMod$;

(Saves the best answers of the model)

$Var \leftarrow X_i$;

(Saves the chosen segmentation variable)

};

};

$rMod_0 \leftarrow \emptyset$;

(Best answers re-initialization)

$rMod_{12} \leftarrow \emptyset$;

(Mod12 answers set initialization)

$C[w] \leftarrow$ Category values of Var ;

$w \leftarrow$ Number of categories of Var ;

For $i = 1$ to w do {

(Loop over the categories of Var)

$S_1 \leftarrow$ Subset of N , such as $Var = C[w]$;

(Creates the two training subsets)

$S_2 \leftarrow$ Subset of N , such as $Var \neq C[w]$;

$rMod_1 \leftarrow$ Regression answers $y \sim \sum_{j=1}^P X_j$, with data from S_1 ;

(Trains the models of the two subsets)

$rMod_2 \leftarrow$ Regression answers $y \sim \sum_{j=1}^P X_j$, with data from S_2 ;

$rMod_{12} \leftarrow rMod_1 \cup rMod_2$;

(Joins the answers of the models of the segments)

If $fMet(rMod_{12}) > fMet(rMod_0)$ {

(If the segmented models improve the result)

$rMod_0 \leftarrow rMod_{12}$;

(Saves the best answers of the models)

$Cat \leftarrow C[w]$;

(Saves the chosen segmentation category)

};

};

Output: Var, Cat ;

(Best segmentation variable and category)

Algorithm 1. *FBTSeg* Pseudo-code

Decision trees aim to separate the population in distinct homogeneous groups regarding the target variable behavior. This strategy was motivated by the fact that these distinct groups often present large differences in the regression parameters estimation and these distinct behaviors may be used as the diversity factor needed in the creation of multiple classifiers.

In this work, the *TradSeg* implementation and the used parameters were similar to those of *FBTSeg*. The main difference is the criterion adopted for splitting the dataset which, in this case, is *information gain*. In other words, the selected variable for the segmentations in each node is the one with more information gain.

III. DATASETS

The used datasets contain different characteristics considered important for the analysis – three of them have been adapted from other published experiments.

In all datasets, used sample sizes are 1,000, 3,000, and 5,000; the training, validation, and test samples are 50%, 25% and, 25% the size of the complete sample, respectively; the following formulae always holds; the target variable (Y) is dichotomic; and h , which is the equation used to generate the dataset, has a different definition for each dataset. Also, it is important to notice that all the datasets contain 50% of examples with $Y=1$ and 50% with $Y=0$.

$$Y = \begin{cases} 1, & \text{if } \frac{1}{1 + \exp(h)} > 0,5 \\ 0, & \text{if } \frac{1}{1 + \exp(h)} \leq 0,5 \end{cases}$$

In addition, in the equations defining h for each dataset, μ_i is an arbitrarily chosen constant, $\varepsilon_i \sim N(0,1)$ is the noise, and $x_1, x_2, \dots, x_{10}$ are variables.

The first dataset has discrete random variables that assume values ranging from two to five categories, varying according to a mass distribution function. In the other three datasets, the variables are random with density $U(0,1)$.

Dataset 1 – Strong interaction on predictive variables: this kind of relationship between predictive and target variables indicates the segmentations should be able to significantly improve the base results. On the other hand, simple linear models should not be able capture all the relationships among the predictive and target variables without explicitly adding new variables.

In this dataset, h was defined as follows:

$$h = \mu_1 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \beta_5 x_5 + \beta_7 x_7 + \beta_8 x_8 + \beta_9 x_9 + \beta_{10} x_{10} + \gamma_1 x_1 x_6 + \gamma_1 x_1 x_8 + \gamma_2 x_2 x_8 + \gamma_3 x_2 x_9 + \gamma_4 x_5 x_7 + \gamma_5 x_6 x_7 + \alpha_1 x_4 x_5 x_6 + \alpha_2 x_8 x_9 x_{10} + \alpha_3 x_2 x_4 x_8 + \varepsilon_1$$

Dataset 2 – A different non-linear structure: this dataset, adapted from [14], is another example with strong non-linearity. The value of h is defined as:

$$h = \mu_2 + 10 \sin(\pi x_1 x_2) + 20(x_3 - 0,5)^2 + 10x_4 + 5x_5 + \varepsilon_2$$

In this dataset, most combination methods should also present significant improvements in the results.

Dataset 3 – Additive effects (linear): in this dataset, adapted from [3], the value of the target variable is only affected by a sum of factors. For the experiments, the samples were generated with h defined by:

$$h = \mu_3 + \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + \omega_4 x_4 + \varepsilon_3$$

In this dataset, with simple dependency, the base classifiers should return good results; so, the ensembles should not improve the results significantly.

Dataset 4 – Quadratic effects: This dataset, also adapted from [3], was generated with h defined as:

$$h = \mu_4 + \sigma_1 x_1^2 + \sigma_2 x_2^2 + \sigma_3 x_3^2 + \sigma_4 x_4^2 + \varepsilon_4$$

Learners that do not consider the quadratic effect of the variables might not capture all the existing dependency between target and predictive variables.

IV. EXPERIMENTS DESIGN AND RESULTS

Linear regression, logistic regression, and neural networks were used as base classifiers (control) to be compared with the combination methods. Significance tests are executed when a combined method has results that are better than those of the base classifier, to make sure they are statistically different.

For all datasets, we created 30 replications of each experiment (k), to calculate confidence intervals and hypotheses tests. Each replication used a separate dataset; the objective was to compare the performance of the combination methods in controlled datasets of different sizes.

Since all numeric variables of the four datasets were created to be continuous in the $[0, 1]$ interval, no preprocessing was needed for training the classifiers. However, because our implementations of both TradSeg and FBTseg only accept categorical variables in their splitting processes, for each numeric variable, we created four categories, each with approximately 25% of the data. It is worth emphasizing that these categorical variables were only used to split data.

A. Defining Experiment Parameters

In both statistic regressions, it was not necessary to choose variables or parameters because the datasets have been generated so that the target variable had a significant relationship with all predictive variables.

In the MLP neural network, we tested all the possible combinations of the two training algorithms (backpropagation and Levenberg-Marquardt) with 3, 10, and 20 neurons in *one* hidden layer. The chosen activation function was sigmoid logistic function having the minimum squared error on the cross-validation set as the training stopping criterion. We used the standard learning rate of SAS Enterprise Miner [17], version 4.0, which cannot be modified in this version of the software. We then chose the best configuration for the classifier (single or combined) in the group of experiments.

For bagging, the chosen size for the re-samples was the size of the original dataset, but allowing repetitions, which is very common [9]. Based on pre-experiments, for both bagging and boosting, the chosen number of re-samples was 25, the same number used in [21].

For the segmentation methods, we used 3 (three) as the maximum depth of the tree because, in preliminary tests, we observed that the best segmentations were in these levels. For the minimum size of a leaf node we chose 10%, a size we judged appropriate, considering the sizes of the datasets. The metric used to evaluate the results of the experiments was the classification error.

It is important to emphasize that our implementation of the segmentation methods used a validation dataset to determine if there was a gain (or not) in each segmentation possible. The validation dataset was 25% of the complete dataset and the same sample was used in the validation of the MLP.

B. Results

Tables I to IV summarize the average classification errors (percentages) and confidence intervals (at 95%) for the single classifiers, crossed with the combination methods and the sample sizes, in the four data sets tested, respectively. The comparison is always based on the control classifier (Simple) of each technique. Average errors that are smaller than that of the respective base classifier are written in bold. Smaller and statistically different values are marked with an asterisk (*). We used the t-Student test for paired data, with significance level of 5%.

In dataset 1 (Table I), using bagging and boosting improved *all* the results when combined with MLP but were not as efficient with the statistic regressions.

TABLE I. SUMMARY RESULTS OF DATASET 1

Size	Tech	Base	Bagging	Boosting	TradSeg	NNTree	FBTSeg
1k	MLP	10,05 $\pm$ 0,73	7,89 $\pm$ 0,58*	9,17 $\pm$ 0,67*	9,87 $\pm$ 0,74	9,80 $\pm$ 0,71	10,01 $\pm$ 0,69
	Linear	18,31 $\pm$ 1,80	17,33 $\pm$ 1,67*	17,17 $\pm$ 1,69*	12,04 $\pm$ 0,77*	12,08 $\pm$ 0,87*	11,11 $\pm$ 0,76*
	Logistic	11,96 $\pm$ 0,74	11,71 $\pm$ 0,69	11,87 $\pm$ 0,71	11,68 $\pm$ 0,67	11,45 $\pm$ 0,67*	9,89 $\pm$ 0,68*
3k	MLP	7,38 $\pm$ 0,34	6,22 $\pm$ 0,38*	6,95 $\pm$ 0,27*	7,48 $\pm$ 0,37	7,39 $\pm$ 0,37	7,54 $\pm$ 0,42
	Linear	16,93 $\pm$ 1,21	16,48 $\pm$ 1,00	16,30 $\pm$ 0,93*	11,20 $\pm$ 0,43*	11,72 $\pm$ 0,47*	8,53 $\pm$ 0,39*
	Logistic	11,28 $\pm$ 0,42	11,14 $\pm$ 0,41	11,19 $\pm$ 0,41	11,03 $\pm$ 0,41	11,08 $\pm$ 0,45*	7,42 $\pm$ 0,35*
5k	MLP	6,75 $\pm$ 0,31	5,57 $\pm$ 0,27*	5,77 $\pm$ 0,41*	6,72 $\pm$ 0,32	6,65 $\pm$ 0,32	6,75 $\pm$ 0,25
	Linear	18,44 $\pm$ 1,68	18,23 $\pm$ 1,60	18,25 $\pm$ 1,64	10,85 $\pm$ 0,32*	11,59 $\pm$ 0,28*	8,02 $\pm$ 0,25*
	Logistic	11,11 $\pm$ 0,31	10,98 $\pm$ 0,27*	11,01 $\pm$ 0,28*	10,41 $\pm$ 0,35*	10,86 $\pm$ 0,30*	7,03 $\pm$ 0,24*

On the other hand, none of the segmentations was significantly better when using MLP as base classifier. However, they improved most results using the statistic regressions – the only exceptions were in the TradSeg combination with logistic regression.

In the specific case of linear regression as the base learner, the segmentations dramatically improved all the results. In particular, *FBTSeg* was also significantly better than the other methods in all sample sizes.

Results presented in Table II are not much different from those of the previous dataset. Again, bagging and boosting have done well with MLP – despite one of the results not being significantly better than that of the base learner. With the statistic regressions they became even worse – no result was significantly improved.

TABLE II. SUMMARY RESULTS OF DATASET 2

Size	Tech	Base	Bagging	Boosting	TradSeg	NNTree	FBTSeg
1k	MLP	10,15 $\pm$ 0,70	8,72 $\pm$ 0,73*	9,84 $\pm$ 0,70	10,61 $\pm$ 0,61	9,58 $\pm$ 0,69	10,36 $\pm$ 0,63
	Linear	17,52 $\pm$ 1,58	17,36 $\pm$ 1,43	17,40 $\pm$ 1,43	13,92 $\pm$ 1,24*	11,84 $\pm$ 0,76*	13,17 $\pm$ 0,88*
	Logistic	14,61 $\pm$ 0,80	14,53 $\pm$ 0,75	14,56 $\pm$ 0,77	14,07 $\pm$ 0,81	11,18 $\pm$ 0,91*	12,91 $\pm$ 0,92*
3k	MLP	7,77 $\pm$ 0,30	7,11 $\pm$ 0,28*	7,26 $\pm$ 0,34*	7,76 $\pm$ 0,31	7,81 $\pm$ 0,34	8,22 $\pm$ 0,37
	Linear	16,24 $\pm$ 0,88	16,31 $\pm$ 0,79	16,26 $\pm$ 0,79	12,88 $\pm$ 0,53*	11,99 $\pm$ 0,43*	12,28 $\pm$ 0,45*
	Logistic	14,79 $\pm$ 0,46	14,72 $\pm$ 0,43	14,75 $\pm$ 0,44	13,00 $\pm$ 0,59*	10,91 $\pm$ 0,40*	12,25 $\pm$ 0,50*
5k	MLP	6,93 $\pm$ 0,27	6,48 $\pm$ 0,27*	6,44 $\pm$ 0,45*	6,99 $\pm$ 0,27	6,93 $\pm$ 0,27	7,30 $\pm$ 0,32
	Linear	15,87 $\pm$ 0,59	15,83 $\pm$ 0,63	15,81 $\pm$ 0,61	12,20 $\pm$ 0,56*	11,38 $\pm$ 0,39*	11,51 $\pm$ 0,53*
	Logistic	14,44 $\pm$ 0,38	14,40 $\pm$ 0,36	14,38 $\pm$ 0,37	12,18 $\pm$ 0,40*	10,42 $\pm$ 0,37*	11,26 $\pm$ 0,49*

One more time, the segmentations failed to improve results significantly using MLP but were very efficient using the statistic regressions. The segmentations also made the results

with both regressions come much closer to those of the MLP base classifier. Both *FBTSeg* and *NNTree* were significantly better in all sample sizes.

Notice that, in all ensembles using this dataset, the classification errors of the statistic regressions were not much affected by the sample size variation. In addition, MLP results were frequently the best and the sample size had significant effect on them, behavior that was also observed and discussed by Kiang [3].

Results of Table III confirm that, because dataset 3 only has simple dependencies, the ensembles were not very efficient. In fact, in very few configurations the results were significantly better than those of the corresponding base classifiers and several results were indeed worse.

TABLE III. SUMMARY RESULTS OF DATASET 3

Size	Tech	Base	Bagging	Boosting	TradSeg	NNTree	FBTSeg
1k	MLP	7,29 $\pm$ 0,70	6,83 $\pm$ 0,58*	7,36 $\pm$ 0,63	7,37 $\pm$ 0,58	7,36 $\pm$ 0,67	7,77 $\pm$ 0,60
	Linear	7,95 $\pm$ 0,77	7,93 $\pm$ 0,83	8,03 $\pm$ 0,86	6,93 $\pm$ 0,58*	7,87 $\pm$ 0,71	7,33 $\pm$ 0,50
	Logistic	6,43 $\pm$ 0,53	6,33 $\pm$ 0,44	6,31 $\pm$ 0,48	6,52 $\pm$ 0,50	6,60 $\pm$ 0,64	6,45 $\pm$ 0,56
3k	MLP	6,60 $\pm$ 0,29	6,38 $\pm$ 0,29*	6,47 $\pm$ 0,29	6,74 $\pm$ 0,31	6,76 $\pm$ 0,30	6,69 $\pm$ 0,30
	Linear	7,20 $\pm$ 0,45	7,09 $\pm$ 0,46	7,04 $\pm$ 0,44*	6,47 $\pm$ 0,24*	7,20 $\pm$ 0,45	6,64 $\pm$ 0,31*
	Logistic	6,33 $\pm$ 0,28	6,22 $\pm$ 0,28*	6,23 $\pm$ 0,27*	6,34 $\pm$ 0,30	6,39 $\pm$ 0,28	6,42 $\pm$ 0,30
5k	MLP	6,66 $\pm$ 0,33	6,51 $\pm$ 0,29*	6,56 $\pm$ 0,30	6,70 $\pm$ 0,33	6,73 $\pm$ 0,34	6,77 $\pm$ 0,29
	Linear	7,40 $\pm$ 0,37	7,35 $\pm$ 0,36	7,35 $\pm$ 0,37	6,74 $\pm$ 0,30*	7,39 $\pm$ 0,35	6,69 $\pm$ 0,31*
	Logistic	6,52 $\pm$ 0,29	6,47 $\pm$ 0,28	6,50 $\pm$ 0,28	6,53 $\pm$ 0,29	6,56 $\pm$ 0,27	6,55 $\pm$ 0,26

In the specific case of the segmentation methods (and to some extent boosting), which are appropriate to capture complex structures, the absence of complexity may cause loss of performance. In the segmentations, irrelevant subgroups may be generated because the same linear structure is kept in the segments.

Experiments summarized in Table IV show that, for a dataset with quadratic effects, the results were once again similar to those of the first two datasets, i.e., bagging and boosting were better with MLP and the segmentations were better with the statistic regressions.

TABLE IV. SUMMARY RESULTS OF DATASET 4

Size	Tech	Base	Bagging	Boosting	TradSeg	NNTree	FBTSeg
1k	MLP	7,24 $\pm$ 0,47	6,31 $\pm$ 0,65*	6,56 $\pm$ 0,60*	7,35 $\pm$ 0,48	7,38 $\pm$ 0,51	7,41 $\pm$ 0,52
	Linear	12,01 $\pm$ 0,73	11,92 $\pm$ 0,72	12,00 $\pm$ 0,72	8,61 $\pm$ 0,65*	8,40 $\pm$ 0,57*	8,48 $\pm$ 0,69*
	Logistic	8,36 $\pm$ 0,69	8,39 $\pm$ 0,71	8,37 $\pm$ 0,68	8,11 $\pm$ 0,69	7,83 $\pm$ 0,70*	8,12 $\pm$ 0,48
3k	MLP	6,56 $\pm$ 0,30	6,51 $\pm$ 0,33	6,51 $\pm$ 0,34	6,60 $\pm$ 0,30	6,45 $\pm$ 0,31	6,82 $\pm$ 0,32
	Linear	11,90 $\pm$ 0,94	11,86 $\pm$ 0,91	11,85 $\pm$ 0,94	8,21 $\pm$ 0,42*	7,69 $\pm$ 0,41*	7,47 $\pm$ 0,38*
	Logistic	8,59 $\pm$ 0,35	8,48 $\pm$ 0,33*	8,51 $\pm$ 0,34	8,38 $\pm$ 0,40*	7,49 $\pm$ 0,31*	7,21 $\pm$ 0,33*
5k	MLP	6,34 $\pm$ 0,20	6,19 $\pm$ 0,22*	6,18 $\pm$ 0,21*	6,39 $\pm$ 0,25	6,27 $\pm$ 0,27	6,51 $\pm$ 0,21
	Linear	11,88 $\pm$ 0,68	11,92 $\pm$ 0,66	11,86 $\pm$ 0,66	7,51 $\pm$ 0,32*	7,57 $\pm$ 0,30*	7,23 $\pm$ 0,24*
	Logistic	8,04 $\pm$ 0,23	8,04 $\pm$ 0,23	8,06 $\pm$ 0,24	7,53 $\pm$ 0,28*	7,24 $\pm$ 0,27*	6,89 $\pm$ 0,27*

V. CONCLUSIONS AND FUTURE WORK

This paper reported implementations of two recent segmentation methods (*FBTSeg* and *NNTree*) and presented the results of experiments comparing them to three established ensemble methods, using three different base classifiers, and three sample sizes in each of four artificially created datasets.

Our main conclusions are:

- Creating ensembles is only worth it when the dataset has complex dependencies;
- Bagging and boosting can improve the performance of MLP neural network learners;
- Segmentation methods can be very efficient to improve the performance of linear regression and logistic regression classifiers; and
- Both *FBTSeg* and *NNTree* are, in most cases, more predictive than the segmentation by information gain.

The *FBTSeg* method improved the results of the statistic regressions as base classifiers in all configurations using datasets 1, 2, and 4, and achieved the top results with both regressions in dataset 1. On the other hand, its performance was much weaker with MLP as well as in dataset 3.

The performance of *NNTree* was very similar to that of *FBTSeg* in most configurations. Even though *NNTree* was not originally developed to work with the statistic regressions, it has also improved the results of these base classifiers in all configurations using datasets 1, 2, and 4, even achieving the top results with both regressions in dataset 2. However, in dataset 3, it did not significantly improve any results at all.

Other experiments with the same segmentation methods and the same base classifiers using a number of datasets of the UCI repository [22] were recently reported [11] and yielded similar results. Specifically, the segmentations methods did not significantly improve the results of experiments using MLP as the base classifier.

Notice that, in most cases, the results using MLP as base learner were much better than those using linear regression, a classifier known to be weaker [3]. However, the use of *FBTSeg* (and of the other segmentation methods too) often led to results that are much closer to the base results obtained with MLP.

An advantage of using segmentation, not discussed yet, is preserving the explanatory characteristic of the technique used to generate the scores. This happens because segmentation methods only use one model per prediction whereas both bagging and boosting can use several. So, explaining the scores only depends on the techniques used.

For example, linear regression can be easily explained since its result is based on a linear equation. On the other hand, the results obtained with neural networks are, in general, harder to explain. This fact and the indication that segmentations together with linear regression might achieve results that are similar to those obtained with MLP as base learner suggest that segmentations could be a viable choice for problems where explaining the results is an essential requirement.

Future work includes studying the behavior of the segmentation methods using other learners such as KNN [5], SVM [5], etc. These and others can be tested either individually or using different techniques in the same training. Another possible direction is to compare the three segmentation methods with other ensemble platforms such as the ones presented in [6].

REFERENCES

- [1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling Wine Preferences by Data Mining from Physicochemical Properties", *Decision Support Systems*, Elsevier, Volume 47, Number 4, pp. 547-553, 2009.
- [2] P. Adeodato, G. Vasconcelos, A. Arnaud, R. Santos, R. Cunha, and D. Monteiro, "Neural Networks vs. Logistic Regression: a Comparative Study on a Large Data Set", In *Proceedings of ICPR'04*, Vol. 3, pp. 355-358, 2004.
- [3] M. Y. Kiang, "A Comparative Assessment of Classification Methods", *Decision Support Systems*, Volume 35, Number 4, pp. 441-454, 2003.
- [4] D. H. Wolpert, "Stacked Generalization", *Neural Networks*, Volume 5, Number 2, pp. 241-259, 1992.
- [5] I. H. Witten and E. Frank, "Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations", Morgan Kaufmann, 2005.
- [6] T. Woloszynski and M. Kurzynski, "A Measure of Competence based on Randomized Reference Classifier for Dynamic Ensemble Selection", In *Proceedings of ICPR'10*, pp. 4194-4197, 2010.
- [7] K. Bock and D. Poel, "An Empirical Evaluation of Rotation-based Ensemble Classifiers for Customer Churn Prediction", *Expert Systems with Applications*, Volume 38, Number 10, pp. 12293-12301, 2011.
- [8] G. Brown, J. Wyatt, R. Harris, and X. Yao, "Diversity Creation Methods: A Survey and Categorisation", In *Journal of Information Fusion*, Volume 6, Number 1, pp. 5-20, 2005.
- [9] J. R. Quinlan, "Bagging, Boosting and C4.5", In *proceedings of the 13th National Conference on Artificial Intelligence*, pp. 725-730, AAAI Press, 1996.
- [10] O. Maimon, and L. Rokach, "Decomposition Methodology for Knowledge Discovery and Data Mining: Theory and Applications", *Series in Machine Perception and Artificial Intelligence*, Volume 61, World Scientific Pub. Inc., 2005.
- [11] R. A. F. Santos and R. S. M. Barros, "Comparing Segmentation Methods with Different Base Classifiers", In *Proceedings of International Conference on Artificial Intelligence (ICAI)*, Las Vegas, NV, 2012.
- [12] R. A. F. Santos, "Um Método para Segmentação de Preditores", Tese de Doutorado, Centro de Informática, UFPE, 2010, In Portuguese, available at <http://www.cin.ufpe.br/~roberto/AlunosPG/Teses/2010-PhD-Roberto.zip>.
- [13] P. Maji, "Efficient Design of Neural Network Tree using a New Splitting Criterion", *Neurocomputing*, Vol. 71, Number 4-6, pp. 787-800, 2008.
- [14] L. C. Thomas, D. B. Edelman, and J. N. Crook, "Credit Scoring and its Applications", Society for Industrial and Applied Mathematics, Philadelphia, 2002.
- [15] R. Johnson and D. Wichern, "Applied Multivariate Statistical Analysis", Prentice Hall, NJ, Sixth Edition, 2007.
- [16] S. Haykin, "Neural Networks and Learning Machines", Third Edition, Prentice Hall, NJ, 2009.
- [17] SAS Institute Inc., "Finding the Solution to Data Mining: a Map of the Features and Components of SAS Enterprise Miner Software", SAS Institute Inc., White paper, Cary, North Carolina, 2000.
- [18] W. J. Conover, "Practical Nonparametric Statistics", Third Edition, John Wiley&Sons, 1999.
- [19] T. Fawcett, "An Introduction to ROC Analysis", *Pattern Recognition Letters*, Volume 27, Number 8, pp. 861-874, 2006.
- [20] P. G. Neville, "Growing Trees for Stratified Modeling", In *Proceedings of 30th Symposium on the Interface: Computing Science and Statistics (Interface 98)*, pp. 528-533, 1998.
- [21] E. Bauer, and R. Kohvi, "An Empirical Comparison of Voting Classification Algorithms: Bagging, Boosting, and Variants", *Machine Learning*, Volume 36, pp. 105-139, 1999.
- [22] C. Blake, and C. Merz, "UCI Repository of Machine Learning Databases", School of Information and Computer Sciences, University of California, Irvine, 2012. <http://archive.ics.uci.edu/ml/>